

Mine spørsmål er markert i gult, alt annet er svar fra ChatGPT

Hvordan hente ut fra en yml fil i en slik oppgave: You are building a quick script to check laboratory equipment calibration status. You will read target settings from a YAML file, look up sensor locations in an Excel sheet, process calibration logs from a CSV file, and output the out-of-date sensors to a JSON file. Write a script check_sensors.py that executes the following pipeline: • Read settings: o Read config.yml and get max_days_since_calibration and output_file

Hvis oppgaven sier at du skal **lese verdier fra en YAML-fil**, er den vanligste løsningen å bruke yaml-biblioteket.

Hvis config.yml for eksempel ser slik ut:

```
max_days_since_calibration: 180
output_file: outdated_sensors.json
```

kan du hente ut verdiene slik:

```
import yaml

with open("config.yml", "r") as file:
    config = yaml.safe_load(file)

max_days = config["max_days_since_calibration"]
output_file = config["output_file"]
```

Da blir:

max_days

lik 180, og

output_file

lik "outdated_sensors.json".

Hva skjer her?

yaml.safe_load(file) leser YAML-filen og gjør den om til en **Python dictionary**:

```
{  
  "max_days_since_calibration": 180,  
  "output_file": "outdated_sensors.json"  
}
```

Derfor kan du hente verdiene med dictionary-syntaks:

```
config["max_days_since_calibration"]  
config["output_file"]
```

Hvis yaml ikke er installert

I terminalen:

```
pip install pyyaml
```

Merk at pakken heter **PyYAML**, men importeres som:

```
import yaml
```

Så for akkurat denne delen av oppgaven ville jeg skrevet:

```
import yaml
```

```
with open("config.yml", "r") as file:  
    config = yaml.safe_load(file)
```

```
max_days = config["max_days_since_calibration"]  
output_file = config["output_file"]
```

Dette er sannsynligvis det læreren forventer at du bruker.

hva menes med "target settings" i yaml filen?

Med **"target settings"** mener de rett og slett **innstillingene/grenseverdiene som programmet skal bruke for å avgjøre hvilke sensorer som er utdaterte.**

I oppgaven står det:

"Read target settings from a YAML file"

og så spesifiseres det hvilke innstillinger:

max_days_since_calibration og output_file

Så hvis config.yml for eksempel inneholder:

max_days_since_calibration: 180

output_file: outdated_sensors.json

er dette **target settings**:

- max_days_since_calibration: 180 → En sensor skal regnes som utdatert hvis det har gått mer enn 180 dager siden kalibrering.
- output_file: outdated_sensors.json → Dette er filnavnet programmet skal lagre resultatet i.

Altså er "target settings" ikke noe spesielt YAML-begrep. Det betyr bare **verdiene som er definert i konfigurasjonsfilen og som programmet skal bruke**.

I Python henter du dem ut slik:

```
max_days = config["max_days_since_calibration"]
```

```
output_file = config["output_file"]
```

Så resten av programmet kan bruke max_days når det sjekker kalibreringsdatoene.

hvordan kan jeg lese data fra de to xlsx og csv filene og kombinere sensor og kalibrering? calibrations-fila har sensor_id, kan jeg skrive noe som sjekker om id matcher med id i xlsx fila, og så legge de matchende sammen?

Ja, akkurat. Det du beskriver er en **join/merge** mellom de to DataFrame-ene, der sensor_id er nøkkelen.

Hvis sensors.xlsx for eksempel ser slik ut:

sensor_id	location
S001	Lab A
S002	Lab B
S003	Lab C

og calibrations.csv:

sensor_id	calibration_date
-----------	------------------

S001	2026-01-10
S002	2025-05-20
S004	2026-02-01

kan du kombinere dem med:

```
combined = pd.merge(sensors, calibrations, on="sensor_id")
```

Da får du bare sensorene som finnes i **begge** filene:

sensor_id	location	calibration_date
S001	Lab A	2026-01-10
S002	Lab B	2025-05-20

Hvorfor fungerer dette?

```
pd.merge(sensors, calibrations, on="sensor_id")
```

betyr omtrent:

Finn rader der `sensor_id` er lik i `sensors` og `calibrations`, og legg informasjonen fra radene sammen.

Du trenger altså **ikke** lage en if-løkke som sjekker hver ID manuelt. `pandas.merge()` gjør dette for deg.

Men pass på hvilken type merge du skal bruke

Dette:

```
pd.merge(sensors, calibrations, on="sensor_id")
```

er en **inner join**, altså bare sensorer som finnes i begge filer.

Du kan også ha:

```
pd.merge(sensors, calibrations, on="sensor_id", how="left")
```

som betyr:

Ta med **alle sensorene fra Excel-filen**, og legg til kalibreringsinformasjon dersom den finnes.

Hva du bør bruke avhenger av resten av oppgaven.

Hvis målet er å **finne sensorer som er ute av dato**, vil det være naturlig å først kombinere dataene og deretter sjekke `calibration_date` mot `max_days_since_calibration`.

Så langt ville jeg tenkt på programmet slik:

`config.yml`

↓

`max_days_since_calibration`

`output_file`

`sensors.xlsx` ─┐

└─→ merge på `sensor_id` → kombinert DataFrame

`calibrations.csv` ┘

↓

sjekk kalibreringsdato

↓

finn utdaterte sensorer

↓

skriv til JSON

Hvis du viser meg **kolonnenavnene i `sensors.xlsx` og `calibrations.csv`**, kan jeg vise deg nøyaktig hvordan merge-delen bør skrives uten å gjøre resten av oppgaven for deg.

sensor fila har `sensor_id`, `lab_room` og `owner`. Oppgaven er: Match each sensor's location/owner with its calibration status.

Ja. Da er det nettopp her `merge()` kommer inn.

Du har i `sensors.xlsx`:

`sensor_id`

`lab_room`

`owner`

og `calibrations.csv` har sannsynligvis noe som:

`sensor_id`

`calibration_date`

Oppgaven:

"Match each sensor's location/owner with its calibration status."

betyr at du skal koble informasjonen fra de to filene via `sensor_id`, slik at du får noe i denne retningen:

<code>sensor_id</code>	<code>lab_room</code>	<code>owner</code>	<code>calibration_date</code>
S001	Lab A	Alice	2026-05-10
S002	Lab B	Bob	2025-12-01

Du trenger altså ikke manuelt sjekke:

```
if sensor_id == ...
```

Bruk `merge()`:

```
combined = pd.merge(sensors, calibrations, on="sensor_id")
```

Her betyr:

```
on="sensor_id"
```

at pandas skal finne samme `sensor_id` i begge filene og koble radene sammen.

Så du kan først gjøre:

```
sensors = pd.read_excel("sensors.xlsx")
calibrations = pd.read_csv("calibrations.csv")
```

```
combined = pd.merge(sensors, calibrations, on="sensor_id")
```

```
print(combined)
```

Resultatet inneholder da `lab_room` og `owner` fra Excel-filen sammen med kalibreringsinformasjonen fra CSV-filen.

Neste steg i oppgaven vil sannsynligvis være å bruke `max_days_since_calibration` fra YAML-filen til å bestemme hvilke av disse som er **out-of-date**.

Resten av oppgaven er: Identify sensors where `days_since_calibration > max_days_since_calibration`. o Export the overdue sensors as a formatted JSON array to the filename specified in `config.yml` using `json.dump(..., indent=2)`. o This json file

should include sensor_id, lab/owner information as well as days_since_calibration for the overdue sensors only. Hva menes med JSON array, og hvordan eksportere som det?

Her betyr **JSON array** egentlig bare en JSON-liste med sensorene som er for gamle.

Hvis du for eksempel har to sensorer som er overdue, skal JSON-filen se omtrent slik ut:

```
[
  {
    "sensor_id": "S001",
    "lab_room": "Lab A",
    "owner": "Alice",
    "days_since_calibration": 250
  },
  {
    "sensor_id": "S003",
    "lab_room": "Lab C",
    "owner": "Charlie",
    "days_since_calibration": 320
  }
]
```

Legg merke til [og]. Det er det som gjør dette til en **JSON array** (liste).

Hvordan lage den i Python

Når du har funnet de overdue sensorene, kan du gjøre dem om til dictionaries og skrive dem til JSON.

For eksempel:

```
import json

overdue = combined[combined["days_since_calibration"] > max_days]

data = overdue[
    ["sensor_id", "lab_room", "owner", "days_since_calibration"]
].to_dict(orient="records")

with open(output_file, "w") as file:
    json.dump(data, file, indent=2)
```

Det viktige her er:

```
.to_dict(orient="records")
```

Dette gjør DataFrame-radene om til en **liste med dictionaries**, altså akkurat strukturen JSON-arrayen trenger.

For eksempel blir:

```
sensor_id | lab_room | owner | days_since_calibration  
S001      | Lab A   | Alice | 250  
S003      | Lab C   | Charlie | 320
```

til omtrent:

```
[  
  {  
    "sensor_id": "S001",  
    "lab_room": "Lab A",  
    "owner": "Alice",  
    "days_since_calibration": 250  
  },  
  {  
    "sensor_id": "S003",  
    "lab_room": "Lab C",  
    "owner": "Charlie",  
    "days_since_calibration": 320  
  }  
]
```

Og så gjør:

```
json.dump(data, file, indent=2)
```

denne Python-listen om til JSON-format.

indent=2 betyr bare at JSON-en skal formateres pent med 2 mellomroms innrykk, slik oppgaven ber om.

Så hele logikken er egentlig:

YAML

↓

max_days_since_calibration

Excel + CSV

↓

merge på sensor_id

↓

beregn days_since_calibration

↓

filtrer: days_since_calibration > max_days

↓

velg sensor_id + lab_room + owner + days_since_calibration

↓

.to_dict(orient="records")

↓

json.dump(..., indent=2)

↓

JSON-fil

En viktig detalj: **du må først beregne days_since_calibration** fra datoen i CSV-filen. Hvis du viser meg kolonnene i calibrations.csv, kan jeg forklare akkurat hvordan den beregningen skal gjøres.

`combined[combined["days_since_calibration"]`

`Hvorfor har du skrevet combined[combined, hvorfor to ganger?`

Det ser litt rart ut, men det er fordi dette er **to forskjellige ting**.

`overdue = combined[combined["days_since_calibration"] > max_days]`

Den innerste delen:

`combined["days_since_calibration"] > max_days`

lager en **True/False-kolonne**. For eksempel:

True

False

True

False

Deretter brukes denne som et filter på combined:

combined[...]

Så hele uttrykket betyr:

Ta combined, men behold bare radene hvor `days_since_calibration > max_days` er True.

Eksempel

Hvis combined er:

sensor_id	days_since_calibration
S001	100
S002	250
S003	300

og:

`max_days = 180`

så gir:

`combined["days_since_calibration"] > max_days`

dette:

False

True

True

Og:

`combined[combined["days_since_calibration"] > max_days]`

gir:

sensor_id	days_since_calibration
S002	250
S003	300

Så **første** `combined[...]` betyr "filtrer denne DataFrame-en", mens **det andre** `combined[...]` henter kolonnen som vi skal bruke til å lage filteret.